Entrada y salida en C

En C, la entrada y salida estándar se manejan con las funciones `printf` (escribir por pantalla) y `scanf` (leer por teclado), ambas definidas en `<stdio.h>`.

```
#include <stdio.h>
```

Escribir por pantalla: `printf`

`printf` recibe una **cadena de formato** seguida de los valores que se insertan en ella. Los **especificadores de formato** indican dónde y cómo se inserta cada valor.

Especificadores de formato

Especificador	Tipo	Ejemplo de uso
`%d`	`int`	`printf("%d", x);`
`%c`	`char`	`printf("%c", letra);`
`%s`	cadena (`char[]`)	`printf("%s", nombre);`
`%f`	`float` / `double`	`printf("%f", nota);`

Secuencias de escape

Secuencia	Significado
`\n`	Salto de línea
`\t`	Tabulador
`\\`	Barra invertida literal
`%%`	Signo `%` literal

Texto fijo

Se puede mezclar texto fijo con especificadores:

```
printf("Hola, mundo\n");
printf("El resultado es %d\n", x);
```

Escribir un entero

```
int x = 42;
printf("El valor de x es %d\n", x);
```

Salida:

```
El valor de x es 42
```

Escribir un carácter

```
char letra = 'A';
printf("La letra es %c\n", letra);
```

Salida:

```
La letra es A
```

Escribir una cadena

```
char nombre[] = "Ada";
printf("Hola, %s\n", nombre);
```

Salida:

```
Hola, Ada
```

Escribir un número real

```
double nota = 7.5;
printf("La nota es %f\n", nota);
```

Salida:

```
La nota es 7.500000
```

Para controlar los decimales, se puede usar `%.2f` (dos decimales):

```
printf("La nota es %.2f\n", nota);
```

Salida:

```
La nota es 7.50
```

Escribir varios valores en un solo `printf`

Cada especificador se corresponde, en orden, con un argumento:

```
int dia = 15, mes = 10, anio = 2026;
printf("Fecha: %d/%d/%d\n", dia, mes, anio);
```

Salida:

```
Fecha: 15/10/2026
```

Escribir campos de un `struct`

Se accede a cada campo con el operador `.` (o `->` si se trabaja con un puntero):

```
typedef struct {
    int fecha;
    int victimas;
} desastre;

desastre d;
d.fecha = 20011203;
d.victimas = 20000;
printf("Fecha = %d; victimas = %d\n", d.fecha, d.victimas);
```

Salida:

```
Fecha = 20011203; victimas = 20000
```

Escribir en un bucle

Es habitual recorrer un vector e imprimir cada elemento:

```
int v[] = {3, 7, 1, 9};
int n = 4;
for (int i = 0; i < n; i++) {
    printf(" %d", v[i]);
}
printf("\n");
```

Salida:

```
 3 7 1 9
```

`printf` sin salto de línea

A veces interesa escribir un mensaje y que el cursor se quede en la misma línea, para que la respuesta del usuario aparezca justo después:

```
printf("Introduce el entero: ");
```

Esto es lo habitual antes de un `scanf`.

Leer por teclado: `scanf`

`scanf` también recibe una cadena de formato, pero en este caso los argumentos son **direcciones de memoria** donde se almacenarán los valores leídos. Para obtener la dirección de una variable se usa el operador `&`.

Leer un entero

```
int x;
printf("Introduce un entero: ");
scanf("%d", &x);
```

Si el usuario escribe `42` y pulsa Intro, `x` valdrá `42`.

Leer un carácter

```
char letra;
printf("Introduce una letra: ");
scanf(" %c", &letra);
```

El espacio delante de `%c` es importante: le dice a `scanf` que descarte cualquier carácter en blanco pendiente (espacios, saltos de línea) antes de leer el carácter. Sin ese espacio, `scanf` podría capturar un `\n` que quedó de una lectura anterior.

Leer una cadena sin espacios

```
char palabra[100];
printf("Introduce una palabra: ");
scanf("%s", palabra);
```

`%s` lee caracteres hasta encontrar un espacio o un salto de línea. Como `palabra` ya es una dirección (el nombre de un array es un puntero a su primer elemento), **no lleva `&`**.

Leer una cadena con espacios

```
char frase[100];
printf("Introduce la frase: ");
scanf("%[^\n]", frase);
```

`%[^\n]` lee todos los caracteres hasta encontrar un salto de línea. Es útil cuando la entrada puede contener espacios, como una frase completa. Al igual que con `%s`, no se usa `&` porque `frase` ya es una dirección.

Leer varios valores a la vez

Se pueden incluir varios especificadores separados por espacios:

```
int fecha, victimas;
printf("Introduce la fecha y las victimas: ");
scanf("%d %d", &fecha, &victimas);
```

El usuario puede separar los valores con espacios o con Intro. Ambas entradas son equivalentes:

```
20011203 20000
```

o:

```
20011203
20000
```

Leer campos de un `struct`

Se usa `&` con cada campo. Si el struct está en un vector, se accede con el índice:

```
typedef struct {
    int fecha;
    int victimas;
} desastre;

desastre vector[10];
int i = 0;
printf("Introduce la fecha y las victimas: ");
scanf("%d %d", &(vector[i].fecha), &(vector[i].victimas));
```

Los paréntesis en `&(vector[i].fecha)` son opcionales pero mejoran la legibilidad: dejan claro que el `&` se aplica al campo entero.

Leer en un bucle

Patrón habitual para llenar un vector:

```
int n;
printf("Introduce el numero de elementos: ");
scanf("%d", &n);

int v[n];
for (int i = 0; i < n; i++) {
    printf("Elemento %d: ", i);
    scanf("%d", &v[i]);
}
```

Leer hasta un valor centinela

En lugar de pedir primero cuántos elementos habrá, se puede leer hasta que el usuario introduzca un valor especial (por ejemplo, `0`):

```
int elem;
printf("Introduce elementos (0 para terminar): ");
scanf("%d", &elem);
while (elem != 0) {
    // procesar elem
    scanf("%d", &elem);
}
```

Patrón del menú

Muchos programas de las prácticas usan un bucle `do-while` con un `switch` para ofrecer un menú al usuario:

```
int opcion;
do {
    printf("--------MENU--------\n");
    printf("1 - Primera opcion\n");
    printf("2 - Segunda opcion\n");
    printf("3 - Salir\n");
    printf("Escoja una opcion: ");
    scanf("%d", &opcion);
    switch (opcion) {
        case 1:
            // ...
            break;
        case 2:
            // ...
            break;
    }
} while (opcion < 3);
```

El menú se repite hasta que el usuario elige la opción de salir.

Resumen de especificadores

Tipo	printf	scanf	scanf lleva & ?
`int`	`%d`	`%d`	Sí
`char`	`%c`	`%c`	Sí
`float` / `double`	`%f`	`%f` / `%lf`	Sí
cadena (sin espacios)	`%s`	`%s`	No
cadena (con espacios)	`%s`	`%[^\n]`	No

Diferencia entre `%f` y `%lf` en `scanf`

En `printf`, tanto `float` como `double` usan `%f`. Pero en `scanf`, `float` usa `%f` y `double` usa `%lf`:

```
float x;
scanf("%f", &x);

double y;
scanf("%lf", &y);
```

Errores frecuentes

Olvidar el `&`

```
int x;
scanf("%d", x);    // MAL: falta &
scanf("%d", &x);   // BIEN
```

Sin `&`, `scanf` interpreta el valor actual de `x` como una dirección de memoria, lo que normalmente provoca un fallo de segmentación.

Capturar un `\n` residual al leer un `char`

Después de un `scanf("%d", ...)`, queda un `\n` en el búfer. Si el siguiente `scanf` es `%c`, capturará ese `\n` en lugar del carácter que el usuario quiere introducir:

```
int n;
scanf("%d", &n);

char letra;
scanf("%c", &letra);   // MAL: letra vale '\n'
scanf(" %c", &letra);  // BIEN: el espacio descarta el '\n'
```

Usar `%s` para cadenas con espacios

```
char frase[100];
scanf("%s", frase);       // Solo lee hasta el primer espacio
scanf("%[^\n]", frase);   // Lee la línea completa
```

Si la entrada es `Hola mundo`, con `%s` solo se lee `Hola`.

Usar `&` con cadenas

```
char nombre[100];
scanf("%s", &nombre);   // Innecesario (aunque suele funcionar)
scanf("%s", nombre);    // Correcto: nombre ya es una dirección
```